Experiment No. 6

Aim

To create, deploy and execute transactions using a Ballot smart contract written in Solidity with Remix IDE.

Theory

A **smart contract** is a program deployed on a blockchain that executes predefined operations. Solidity is a programming language used to develop smart contracts for Ethereum-compatible blockchains. Remix IDE provides an environment to write, compile, deploy and test these contracts.

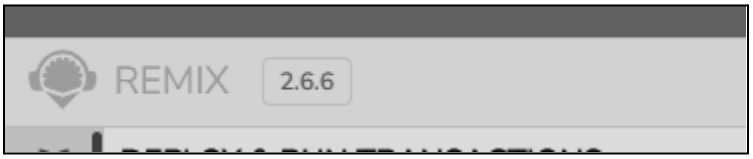
The **Ballot contract** implements a voting system in which a chairperson grants voting rights, voters select proposals, and votes can be delegated to other voters. The contract also determines the winning proposal based on the vote count.

The `require()` statement checks conditions and prevents unauthorized operations or invalid transactions. A **mapping** associates an account address with its voter information. **Storage** holds persistent blockchain data, whereas **memory** holds temporary data during function execution.

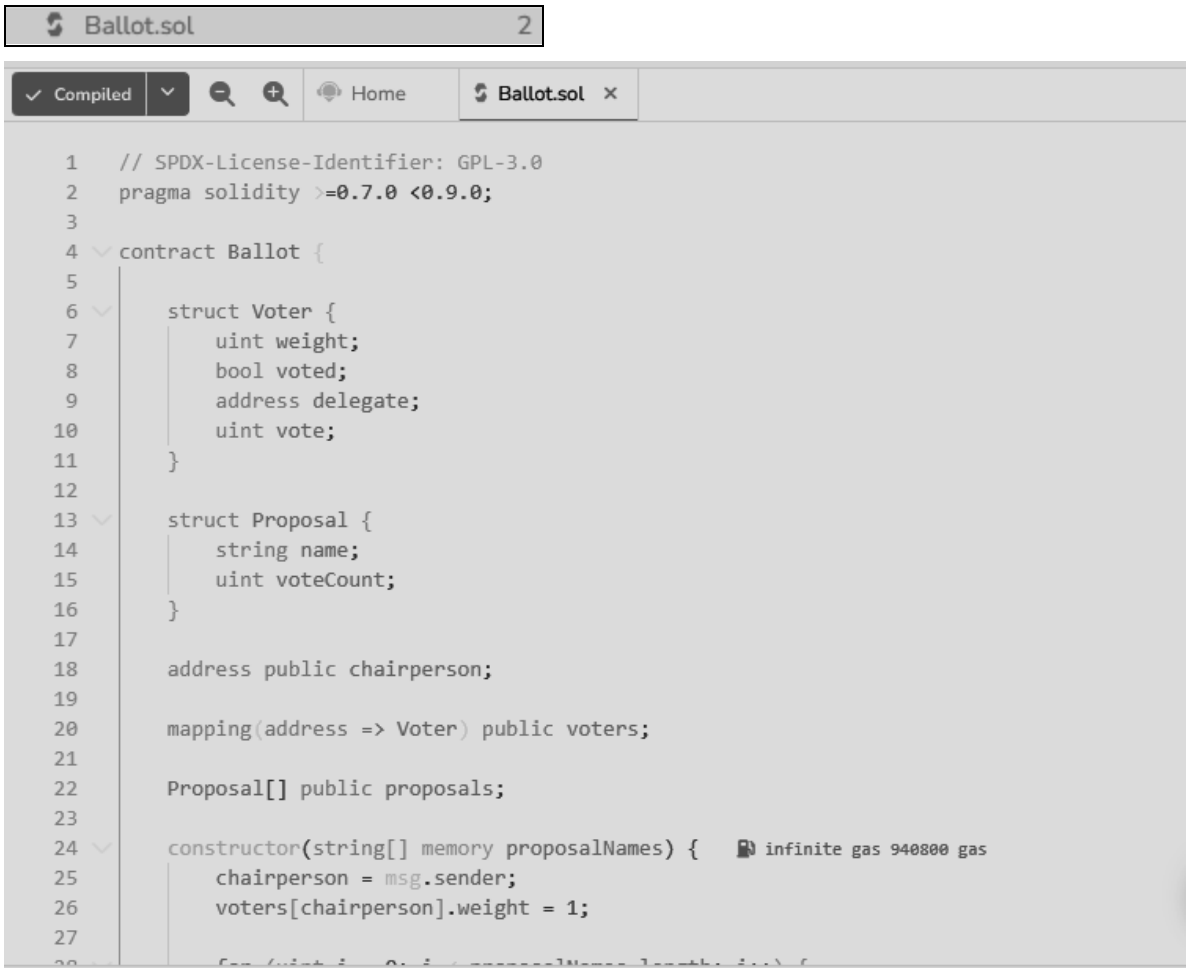
The original contract uses `bytes32` for proposal names because it is a fixed-size data type. In this experiment, it is changed to `string` to support proposal names as ordinary text.

Step-by-Step Procedure

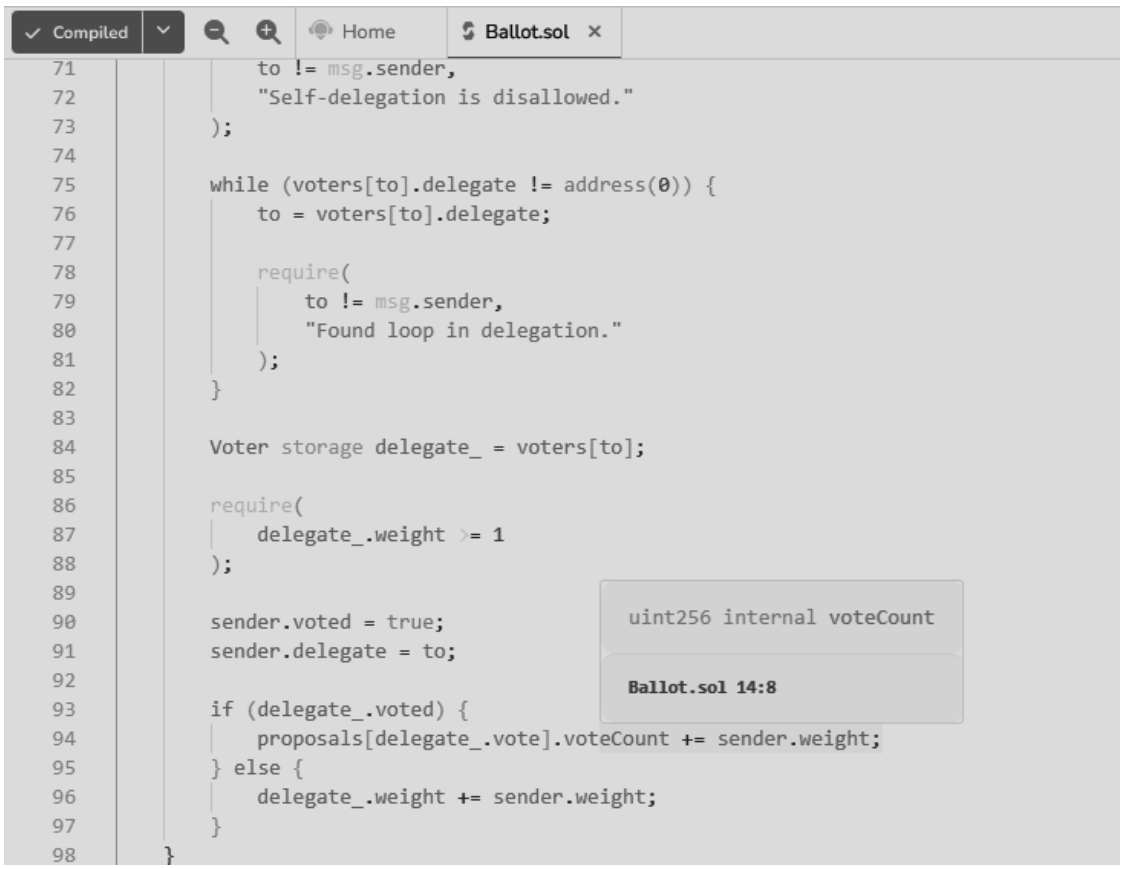
Step 1: Open Remix IDE
Open https://remix.ethereum.org/ and access the File Explorer.



Step 2: Create the Solidity file
Create a file named `Ballot.sol` inside the `contracts` folder and enter the modified Ballot contract code.


Ballot.sol 2
Compiled
Home
Ballot.sol
// SPDX-License-Identifier: GPL-3.0
pragma solidity >=0.7.0 <0.9.0;
contract Ballot {
struct Voter {
uint weight;
bool voted;
address delegate;
uint vote;
}
struct Proposal {
string name;
uint voteCount;
}
address public chairperson;
mapping(address => Voter) public voters;
Proposal[] public proposals;
constructor(string[] memory proposalNames) {
infinite gas 940800 gas
chairperson = msg.sender;
voters[chairperson].weight = 1;

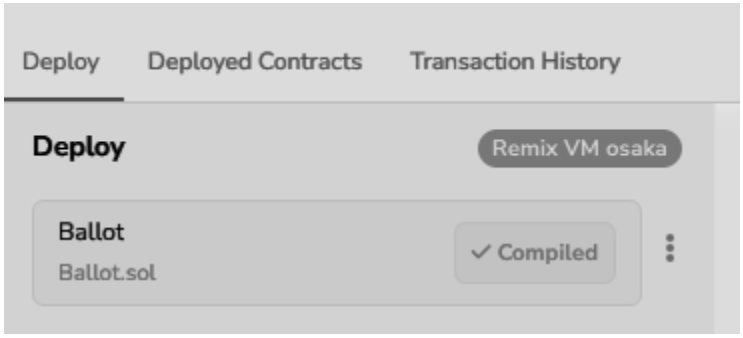

Compiled
Home
Ballot.sol
function giveRightToVote(address voter) external {
29310 gas
require(
msg.sender == chairperson,
"Only chairperson can give right to vote."
);
require(
!voters[voter].voted,
"The voter already voted."
);
require(
voters[voter].weight == 0,
"Voter already has the right to vote."
);
voters[voter].weight = 1;
}
function delegate(address to) external {
infinite gas
Voter storage sender = voters[msg.sender];
require(
sender.weight != 0,
"You have no right to vote"
);


Compiled
Home
Ballot.sol
to != msg.sender,
"Self-delegation is disallowed."
);
while (voters[to].delegate != address(0)) {
to = voters[to].delegate;
require(
to != msg.sender,
"Found loop in delegation."
);
}
Voter storage delegate_ = voters[to];
require(
delegate_.weight >= 1
);
sender.voted = true;
sender.delegate = to;
if (delegate_.voted) {
proposals[delegate_.vote].voteCount += sender.weight;
} else {
delegate_.weight += sender.weight;
}
}
uint256 internal voteCount
Ballot.sol 14:8

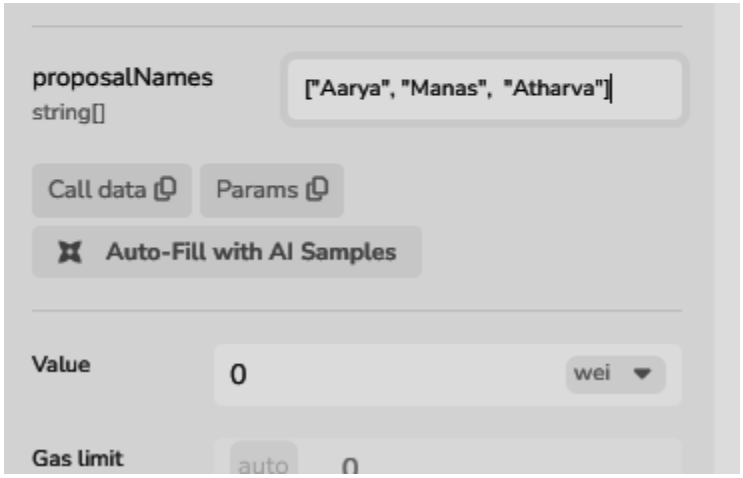
Step 3: Compile the contract
Open Solidity Compiler, select a compatible compiler version and click **Compile Ballot.sol**. Verify that compilation succeeds.



Step 4: Deploy the contract
Open Deploy & Run Transactions, select Remix VM and choose the Ballot contract.

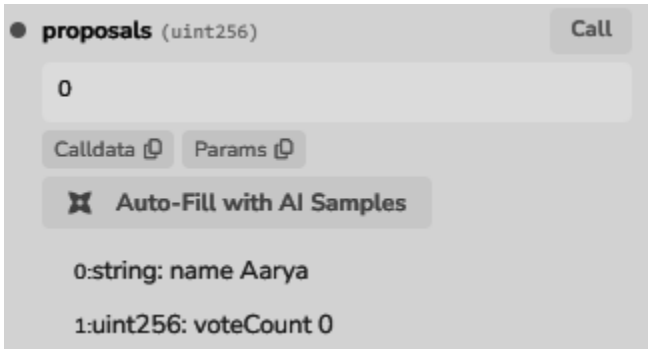


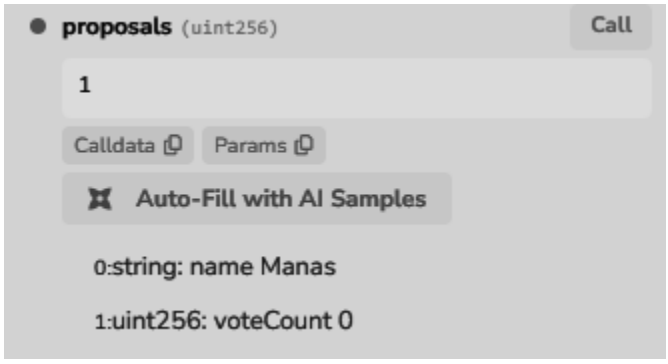
Step 5: Enter proposal names
Enter `["Candidate A","Candidate B","Candidate C"]` in the constructor field and click **Deploy**.

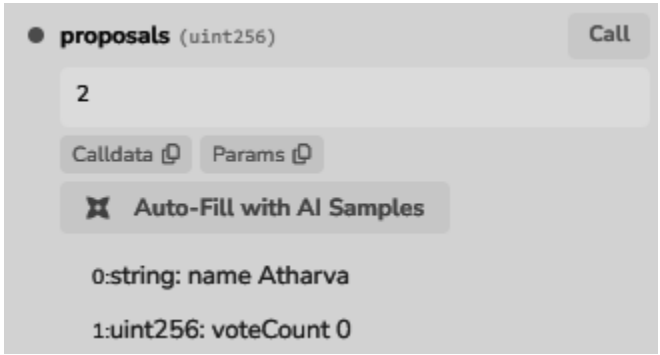


Step 6: Check the proposals
Call `chairperson` to view the deploying account and use `proposals` with indices 0, 1 and 2 to view the candidates.



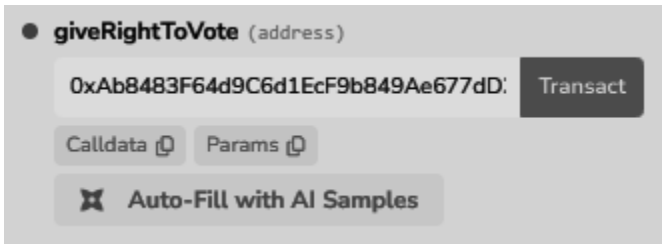






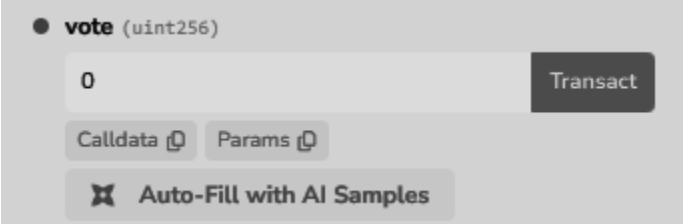
Step 7: Grant voting rights

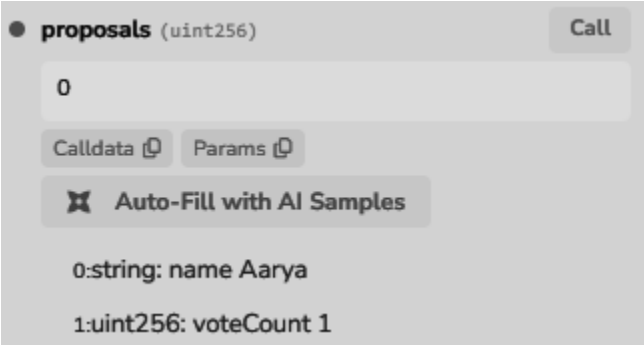
Using the chairperson account, call `giveRightToVote` with another account's address.

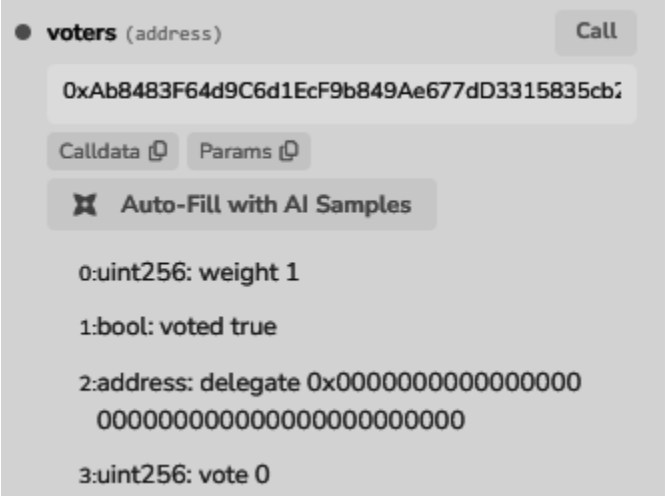


Step 8: Cast a vote

Select the authorized voter account and call `vote` with the proposal index, such as `0` for Candidate A. and also verify the vote given by checking the proposal and voters call.

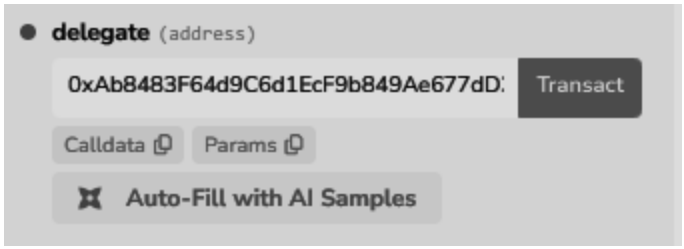


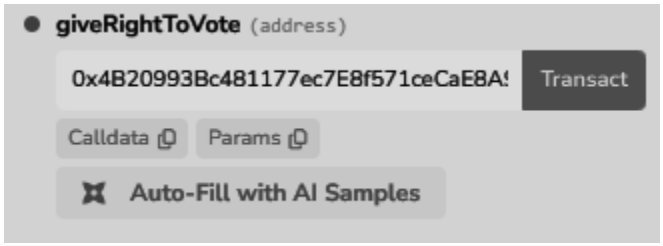




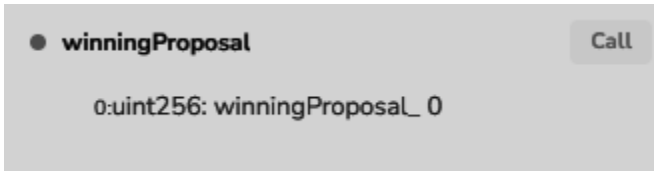
Step 9: Delegate a vote

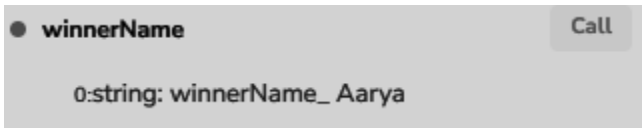
Grant voting rights to another account, then use its `delegate` function to delegate its vote to an authorized voter. Check the voter information and voting weight.





Step 10: Find the winner
Call `winningProposal()` to obtain the winning proposal's index and `winnerName()` to display its name.





Conclusion

Thus, the Ballot smart contract was created, compiled and deployed using Solidity and Remix IDE. Voting rights, voting, delegation and identification of the winning proposal were demonstrated. The experiment also provided an understanding of `require()`, mapping, storage, memory, and the difference between `bytes32` and `string`.